ARQUITECTURA DE SOFTWARE

Taller del 2do Corte — Docker

Parte 1: Investigativa

Fundamentos de Docker: qué es, componentes, funcionalidades, comandos, problema que resuelve e importancia en la Arquitectura de Software

Arquitectura de software 1508

Estudiante: Daniel de Jesús Carbal Castro

T00082732

Profesor: Luis Giovanny Carreno Ortiz

16 de octubre de 2026

Introducción

Hoy en día casi todos los proyectos de software usan contenedores, y Docker es la herramienta más conocida para trabajar con ellos. Lo que hace es permitir que una aplicación se pueda mover de un computador a otro, o de un servidor a otro, y funcione igual en todos. En una arquitectura de software, donde el sistema se divide en varias partes (por ejemplo una API, una base de datos y un broker de eventos), esto ayuda mucho.

En este informe explico qué es Docker, cómo funciona, cuáles son sus componentes, funcionalidades y comandos principales, qué problema vino a resolver y por qué es importante en lo que hemos visto en el curso.

Desarrollo

1. ¿Qué es Docker?

Definición. Docker es una herramienta que permite meter una aplicación, junto con todo lo que necesita para funcionar (el lenguaje, las librerías, la configuración), dentro de una "caja" llamada **contenedor**. Esa caja se puede ejecutar de la misma forma en cualquier computador que tenga Docker instalado, sin importar qué más tenga instalado.

Cómo funciona. Un contenedor no es un computador completo, sino un proceso que corre aislado dentro del sistema operativo de la máquina donde está (el host). El aislamiento lo da el kernel de Linux: hace que el contenedor solo vea sus propios procesos, su propia red y sus propios archivos, y además permite ponerle límites de memoria y de CPU. Como usa el kernel del host, no necesita arrancar un sistema operativo propio; solo trae las librerías y archivos que la aplicación necesita. En Windows y Mac, Docker Desktop usa una máquina virtual pequeña de Linux para tener ese kernel.

Contenedor vs. máquina virtual. Una máquina virtual simula un computador completo y cada una trae su propio sistema operativo, por eso pesa varios gigabytes y se demora minutos en encender. Un contenedor comparte el kernel del host y solo lleva la aplicación con sus dependencias, entonces pesa mucho menos y arranca en segundos. Una forma de verlo es que las máquinas virtuales son como casas separadas, cada una con sus propios cimientos, y los contenedores son como apartamentos de un mismo edificio que comparten la estructura. Por eso en un mismo computador caben muchos más contenedores que máquinas virtuales.

2. ¿Cuáles son sus componentes?

- **CLI (cliente docker):** es la herramienta de línea de comandos que uno usa para escribir las órdenes (docker run, docker build, etc.). No ejecuta los contenedores directamente, solo le envía las órdenes al daemon.
- **Daemon (dockerd):** es el motor de Docker. Corre en segundo plano y es quien hace el trabajo: construye imágenes, crea y ejecuta contenedores y maneja las redes y los volúmenes.
- **Imagen:** es una plantilla de solo lectura que tiene todo lo necesario para correr una aplicación (por ejemplo, Go y el código ya compilado). Es como el molde a partir del cual se crean los contenedores.

- **Contenedor:** es una imagen en ejecución. De una misma imagen se pueden crear muchos contenedores, y cada uno funciona de forma independiente.
- **Dockerfile:** es un archivo de texto con los pasos para construir una imagen (instrucciones como FROM, COPY, RUN y CMD). Es como la receta de la imagen.
- **Registro (registry):** es un lugar donde se guardan y comparten imágenes. El más conocido es Docker Hub, pero también hay registros privados como el de GitHub o el de Amazon.

Cómo se relacionan. El usuario escribe un comando en la CLI, la CLI se lo pasa al daemon, y el daemon trabaja con las imágenes y contenedores que hay en la máquina. Cuando necesita una imagen que no tiene, o quiere publicar una, se conecta al registro:

Usuario → CLI (docker) → Daemon (dockerd) → Imágenes y Contenedores locales ↔ Registro (Docker Hub)

La cadena Dockerfile → Imagen → Contenedor. Primero se escribe el **Dockerfile**. Con el comando docker build, Docker lee ese archivo y crea la **imagen**. Con docker run, Docker toma esa imagen y arranca un **contenedor**. El **registro** entra en dos momentos: con docker pull se descarga una imagen que alguien más ya construyó (por ejemplo mysql:8), sin necesitar su Dockerfile, y con docker push se sube una imagen propia para que otras personas o un servidor la puedan descargar.

Dockerfile —(docker build)→ Imagen —(docker run)→ Contenedor
Registro: docker push (subir) ↔ docker pull (descargar) imágenes

3. ¿Cuáles son sus principales funcionalidades?

- **Portabilidad.** Sirve para que una misma imagen funcione igual en cualquier máquina con Docker: el computador del desarrollador, un servidor o la nube. Por ejemplo, una API hecha en Go dentro de una imagen corre igual en Windows, Mac o Linux, sin tener que instalar Go.
- **Aislamiento.** Cada contenedor tiene sus propios procesos, red y archivos, así que las aplicaciones no se afectan entre sí. Por ejemplo, puedo tener un proyecto con MySQL 5.7 y otro con MySQL 8 en el mismo computador, cada uno en su contenedor, sin que choquen.
- **Versionado de imágenes.** Las imágenes tienen etiquetas (tags) que indican la versión, y eso sirve para elegir exactamente cuál usar y volver a una anterior si algo sale mal. Por ejemplo, mysql:8 o node:18 indican versiones concretas, y si mi versión 1.1 de la API falla puedo volver a biblioteca-api:1.0.
- **Reproducibilidad del entorno.** Como el entorno queda escrito en archivos (Dockerfile y docker-compose.yml), todo el equipo trabaja con lo mismo. Por ejemplo, un compañero nuevo clona el repositorio, ejecuta docker compose up y en pocos minutos ya tiene la API y la base de datos funcionando.
- **Escalabilidad.** Permite crear varias copias del mismo contenedor para repartir la carga. Por ejemplo, si llegan muchos usuarios a la vez, se pueden levantar tres contenedores de la misma API con docker compose up --scale api=3.

4. ¿Cuáles son sus principales comandos?

a) Tabla de comandos

Comando	¿Qué hace?	Ejemplo
docker pull	Descarga una imagen desde un registro (por defecto Docker Hub) a la máquina local.	docker pull mysql:8
docker run	Crea e inicia un contenedor a partir de una imagen; si la imagen no existe localmente, la descarga.	docker run -d --name db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secreto mysql:8
docker ps	Lista los contenedores que están en ejecución.	docker ps
docker ps -a	Lista todos los contenedores, incluidos los detenidos.	docker ps -a
docker logs	Muestra la salida (logs) de un contenedor; con -f se sigue en tiempo real.	docker logs -f db
docker exec	Ejecuta un comando dentro de un contenedor que ya está corriendo.	docker exec -it db mysql -u root -p
docker stop / start	Detiene o reanuda un contenedor sin eliminarlo.	docker stop db docker start db
docker rm	Elimina un contenedor detenido (con -f fuerza la eliminación de uno en ejecución).	docker rm db
docker images	Lista las imágenes almacenadas localmente.	docker images
docker rmi	Elimina una imagen local que ningún contenedor esté usando.	docker rmi mysql:8
docker build	Construye una imagen a partir de un Dockerfile.	docker build -t biblioteca-api:1.0 .
docker compose up	Crea e inicia todos los servicios definidos en el archivo docker-compose.yml.	docker compose up -d --build

b) Tres comandos explicados

docker run. Crea y arranca un contenedor a partir de una imagen. Los parámetros más comunes son: **-d**, que lo deja corriendo en segundo plano; **-p host:contenedor**, que conecta un puerto de mi computador con uno del contenedor (con -p 8080:80 entro por localhost:8080 al puerto 80 del contenedor); **-e VARIABLE=valor**, que define una variable de entorno; **-v origen:destino**, que guarda los datos en un volumen para que no se pierdan; **--name**, que le pone un nombre fácil de recordar; **--rm**, que borra el contenedor cuando se detiene; y **-it**, que abre una sesión interactiva. Lo usaría, por ejemplo, para tener una base de datos de desarrollo sin instalarla en mi computador: docker run -d --name db -p 3306:3306 -e MYSQL_ROOT_PASSWORD=secreto -v datos:/var/lib/mysql mysql:8. El volumen hace que los datos sigan ahí aunque borre el contenedor.

docker build. Construye una imagen a partir de un Dockerfile. Los parámetros más comunes son: **-t nombre:etiqueta**, que le da nombre y versión a la imagen; **-f ruta**, que sirve cuando el Dockerfile tiene otro nombre o está en otra carpeta; **--no-cache**, que obliga a construir todo desde cero sin usar lo guardado de antes; y el **punto final (.)**, que indica la carpeta de trabajo con los archivos que el Dockerfile puede usar. Lo usaría cada vez que cambie el código de mi API y necesite una nueva imagen: docker build -t biblioteca-api:1.1 .

docker compose up. Lee el archivo docker-compose.yml y levanta todos los servicios que están definidos ahí (por ejemplo la API y la base de datos), con su red y sus volúmenes, usando un solo comando. Los parámetros más comunes son: **-d**, para correrlo en segundo plano; **--build**, para reconstruir las imágenes antes de arrancar; **-f archivo**, para usar otro archivo de Compose; y **--scale servicio=N**, para crear N copias de un servicio. Lo usaría siempre que el proyecto tenga más de un contenedor que deba trabajar junto: docker compose up -d --build. Para apagarlo se usa docker compose down (y con -v también se borran los volúmenes).

5. ¿Qué problema resuelve?

Docker apareció para resolver el problema de **"en mi máquina funciona"**. Pasa cuando una aplicación se desarrolla y se prueba en un computador y funciona bien, pero al llevarla a otro (el de un compañero, el servidor de pruebas o producción) falla. Esto ocurre porque una aplicación no depende solo de su código, sino también de todo lo que la rodea. Las causas más comunes son:

- **Versiones de dependencias:** el desarrollador usa una versión de una librería o del lenguaje, y en el servidor hay otra que se comporta distinto.
- **Sistema operativo:** hay diferencias entre Windows, Mac y las distintas versiones de Linux, por ejemplo en las rutas de archivos, los permisos o las librerías del sistema.
- **Configuración:** variables de entorno, puertos u otros parámetros que están definidos en una máquina pero no en la otra.
- **Instalación manual:** los pasos para preparar el entorno dependen de que alguien se los acuerde o de un documento desactualizado, y es fácil saltarse o cambiar un paso.

Ejemplo antes y después. Antes: un equipo desarrolla una API en Go con MySQL. Para correrla, cada persona tiene que instalar Go, instalar MySQL, crear la base de datos, ejecutar los scripts y configurar las variables. Un compañero tiene MySQL 5.7, otro tiene MySQL 8, y a uno de los dos la API le falla por un error de autenticación. Se pierde medio día buscando la causa. **Después:** el proyecto trae un Dockerfile y un docker-compose.yml. Cualquiera clona el repositorio, ejecuta docker compose up y en pocos minutos tiene la API y la base de datos funcionando, con las mismas versiones. Ya no importa qué tiene instalado cada quien, porque el entorno viaja junto con la aplicación.

6. ¿Por qué es importante dentro de una Arquitectura de Software?

Un contenedor por servicio. Conviene que cada parte de la arquitectura (la API, la base de datos, el broker de eventos) corra en su propio contenedor, porque así cada una tiene sus propias dependencias y versiones, se puede actualizar o reiniciar sin tocar las demás, y si una falla no arrastra a las otras. Además, esto va de la mano con la idea de separar responsabilidades: cada pieza hace una sola cosa y se comunica con las demás por la red.

Despliegue consistente. La imagen que se construye es la misma que se prueba y la misma que se despliega, así que el entorno es igual en desarrollo, pruebas y producción. Eso evita los problemas

de “en desarrollo sí funcionaba” y es la base del **CI/CD**, donde un servidor automático construye la imagen, corre las pruebas y la sube a un registro para desplegarla.

Escalabilidad y portabilidad a la nube. Si un servicio recibe más carga, basta con levantar más copias de su contenedor, y herramientas como Kubernetes pueden hacerlo automáticamente. Además, como los contenedores no dependen de la infraestructura, la misma imagen se puede ejecutar en un servidor propio o en cualquier proveedor de nube (AWS, Azure, Google Cloud), sin quedar amarrado a uno solo.

Relación con los estilos arquitectónicos. En **microservicios**, cada servicio es pequeño e independiente y se empaqueta y escala por separado, así que el contenedor es su forma natural de despliegue. En **SOA**, los servicios son autónomos y se comunican por interfaces definidas, y los contenedores ayudan a aislarlos y desplegarlos de forma independiente. En una arquitectura **orientada a eventos**, los productores, los consumidores y el broker (como RabbitMQ o Kafka) pueden correr cada uno en su contenedor, y se pueden agregar más consumidores si aumentan los eventos. Incluso en una arquitectura por capas, como la de **biblioteca-api** (handler → service → repository), Docker sirve para levantar la aplicación con su base de datos de forma repetible.

Conclusión

Docker resuelve un problema muy concreto, que es lograr que el software funcione igual en todas partes, y lo hace de forma liviana porque aprovecha el kernel de la máquina en lugar de simular un sistema operativo completo. Entender la cadena Dockerfile - imagen - contenedor, y el papel del registro, me ayudó a ver que el entorno de una aplicación se puede tratar como código: se escribe, se versiona y se comparte. Eso cambia mucho la forma de desarrollar y desplegar.

Usaría Docker en cualquier proyecto que tenga más de una pieza, por ejemplo una aplicación web con una API y una base de datos como la que he trabajado anteriormente. Así no tendría que instalar y configurar cada herramienta en mi computador, y podría entregar el proyecto para que cualquier persona lo ejecute con un solo comando. Creo que para alguien que va a trabajar en arquitectura de software, saber usar Docker es casi obligatorio, porque es la base de los microservicios, de los sistemas orientados a eventos y de los despliegues en la nube.

Referencias

Docker Inc. (s. f.). *Docker overview*. Docker Docs. https://docs.docker.com/get-started/docker-overview/

Docker Inc. (s. f.). *What is a container?* Docker Docs. https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Docker Inc. (s. f.). *Dockerfile reference*. Docker Docs. https://docs.docker.com/reference/dockerfile/

Docker Inc. (s. f.). *Docker CLI reference*. Docker Docs. https://docs.docker.com/reference/cli/docker/

Docker Inc. (s. f.). *Docker Compose overview*. Docker Docs. https://docs.docker.com/compose/

Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. *Linux Journal*, 2014(239).